

```
graph LR; Q[Question + schema] --> I[Interpret  
intent program  
answer signature]; I --> V{valid intent?}; V -- yes --> C[Compile  
entities · DAG  
executable graph]; V -- no / veto --> P[Step-2 planner  
fallback graph plan]; C --> G[Ground  
schema · values  
guards · preflight]; G --> E[Execute  
bound variables  
exact reduction]; E --> R[Release checks  
evidence · sanity  
postflight]; R --> A[Answer + evidence  
or  
No verified answer]; R -.-> F[structured failure · bounded replan · rerun ground → execute → checks]; F -.-> P;
```

The flowchart illustrates the Step-2 planner process. It begins with a **Question** (including schema) which is passed to the **Interpret** stage (intent program, answer signature). A decision diamond checks if the intent is **valid**. If **yes**, the process continues to **Compile** (entities · DAG, executable graph), then **Ground** (schema · values, guards · preflight), then **Execute** (bound variables, exact reduction), and finally **Release checks** (evidence · sanity, postflight). The **Release checks** stage leads to either **Answer + evidence** or **No verified answer**. A feedback loop from the **Release checks** stage, labeled **structured failure · bounded replan · rerun ground → execute → checks**, feeds into the **Step-2 planner** (fallback graph plan), which then feeds back into the **Compile** stage. If the intent is **no / veto**, the process also feeds into the **Step-2 planner**.

```

graph LR
    A[Record check: how many contracts were published by buyers that have awarded a contract to Dodd Group (Midlands) Limited?] --> B[Step-1 intent program  
A filter supplier = Dodd Group  
B distinct buyers → C filter buyer  
D count(C)]
    B --> C[Compiled graph  
A record_set  
→ B buyer_set  
→ C bound records]
    C --> D[Execute  
count(C)  
no repair]
    D --> E[Verified result  
1,940  
passed · oracle match ✓]
  
```

Valid Step-1 program → deterministic compiler; the conditional Step-2 planner is bypassed for this trace.